Pintos 학습 가이드

버전: 2026-04-25 대상 repo: /workspaces/week09-team-03-p
x86-64 기반 threads, userprog, vm, filesys 전체 학습 로드맵

이 문서는 Pintos를 처음부터 끝까지 한 번에 외우기 위한 문서가 아니다. 코드를 읽는 순
과제별 핵심 개념, 구현할 때 자주 무너지는 지점, 테스트를 해석하는 방법을 연결해서 "지금
내가 어디를 보고 있고 왜 이 파일을 바꾸는지"를 잃지 않게 만드는 학습 자료다.

Pintos는 작은 교육용 OS지만, 만만한 장난감은 아니다. 커널 스레드, 인터럽트, 스케줄
사용자 주소 공간, 시스템콜, 가상 메모리, 파일 시스템이 모두 서로 물려 있다. 그래서 한
기능을 고칠 때도 "자료구조", "락/인터럽트", "수명 관리", "테스트가 기대하는 관찰
가능한 동작"을 동시에 생각해야 한다.

목차

1.공부 방법과 전체 지도
2.repo 구조와 빌드/실행 루틴
3.Pintos 커널의 큰 흐름
4.꼭 알아야 할 C/OS 배경지식
5.Pintos list 사용법
6.Project 1: Threads
7.Project 2: User Programs
8.Project 3: Virtual Memory
9.Project 4: File System
10디버깅과 테스트 읽는 법
11구현 체크리스트
12용어집과 참고 자료

1. 공부 방법과 전체 지도

Pintos를 공부하는 기본 태도

Pintos 공부는 "강의 슬라이드 읽기"보다 "작은 커널을 디버깅하며 설계 의도를 역추적하기"에 가깝다. 처음부터 완성 코드를 떠올리려고 하면 거의 항상 길을 잃는다. 대신 다음 순서를 반복한다.

1. 테스트 이름을 읽는다.
2. 테스트가 요구하는 관찰 가능한 동작을 말로 쓴다.
3. 그 동작을 담당하는 공개 함수의 주석을 읽는다.
4. 공개 함수가 어떤 내부 자료구조를 쓰는지 찾는다.
5. 자료구조의 불변식을 한 문장으로 쓴다.
6. 최소 구현을 한다.
7. 실패 로그를 보고 불변식이 깨진 위치를 좁힌다.

예를 들어 alarm-multiple을 공부한다면 "잠든 스레드는 기다리는 동안 CPU를 쓰지 하고, 충분한 tick이 지난 뒤 ready queue로 돌아와야 한다"가 요구 동작이다. 이 다음 timer_sleep(), timer_interrupt(), thread_block(), 보면 된다.

처음 한 바퀴 읽는 순서

Pintos는 파일이 많지만 처음부터 모두 읽을 필요는 없다. 아래 순서대로 읽으면 커널의 뼈 보인다.

- pintos/threads/init.c: 부팅 뒤 main()이 어떤 초기화를 하는지 본다.
- pintos/threads/thread.c: 스레드 생성, ready list, yield,
- pintos/include/threads/thread.h: struct thread가 무엇을
- pintos/devices/timer.c: timer interrupt와 timer_sleep
- pintos/threads/synch.c: semaphore, lock, condition v 어떻게 감싸는지 본다.
- pintos/include/lib/kernel/list.h: Pintos intrusive l
- pintos/userprog/process.c: 사용자 프로그램 로딩, ELF segment,
- pintos/userprog/syscall.c: 사용자 프로그램이 커널 서비스를 요청하는 입구
- pintos/include/threads/interrupt.h: struct intr_fram
- pintos/include/vm/vm.h와 pintos/vm/vm.c: VM project의
- pintos/filesys/inode.c, pintos/filesys/file.c, pinto object의 수명과 계층을 본다.

과제별 핵심 질문

Project 1 Threads의 핵심 질문:

- 어떤 스레드가 언제 THREAD_READY, THREAD_BLOCKED, THREAD_RUNN
- ready list는 어떤 순서로 유지되어야 하는가?
- 인터럽트 handler 안에서 해도 되는 일과 하면 안 되는 일은 무엇인가?
- priority donation은 "누가 누구에게 왜 priority를 빌려주는가"로 설명할

Project 2 User Programs의 핵심 질문:

- 사용자 가상주소와 커널 주소를 어떻게 구분하는가?
- 시스템콜 인자는 어느 레지스터로 들어오는가?
- 사용자 포인터가 잘못되었을 때 커널이 죽지 않게 하려면 어느 경계에서 검사해야 하는가?
- parent/child process의 lifetime을 어떤 자료구조로 관리할 것인가?

Project 3 Virtual Memory의 핵심 질문:

- page와 frame의 차이는 무엇인가?
- page table과 supplemental page table은 각각 어떤 정보를 맡는가?
- page fault가 "정상적인 lazy loading"인지 "죽여야 할 invalid acc
  판별하는가?
- frame eviction, swap, mmap writeback의 책임이 어디에 있는가?

Project 4 File System의 핵심 질문:

- inode가 disk sector를 어떻게 가리키는가?
- 파일 길이가 커질 때 index block을 어떻게 확장할 것인가?
- directory entry, current working directory, symlink의
- buffer cache가 disk I/O와 동기화를 어떻게 바꾸는가?

추천 학습 루틴

하루 공부를 다음 단위로 자르면 좋다.

- 20분: 공식 매뉴얼에서 오늘 구현할 요구사항만 읽기.
- 30분: 관련 파일의 함수 call graph를 손으로 그리기.
- 60분: 아주 작은 구현 단위 하나만 수정하기.
- 30분: 테스트 하나만 돌리고 실패 로그 읽기.
- 20분: 오늘 새로 배운 불변식 3개를 문장으로 정리하기.

Pintos에서 "불변식"은 정말 중요하다. 예를 들면 thread_block()은 inter
호출되어야 한다. struct list_elem 하나는 동시에 두 리스트에 들어갈 수 없다.
파괴하기 전에는 현재 CPU가 그 page table을 사용하지 않게 해야 한다. 이런 문장들
코드가 덜 무섭다.

2. repo 구조와 빌드/실행 루틴

이 repo의 큰 구조

현재 repo는 KAIST Pintos x86-64 스타일이다. top-level 아래에 pi
테스트, 라이브러리, 사용자 프로그램용 라이브러리가 들어 있다.

- pintos/threads: base kernel, scheduler, synchronizat
- pintos/devices: timer, disk, keyboard, serial 같은 dev
- pintos/userprog: process loader, syscall, exception
- pintos/vm: supplemental page table, frame, anon/file
- pintos/filesys: inode, directory, file, free map, FA
- pintos/lib: kernel과 user program이 공유하는 C library sub
- pintos/include: header files.
- pintos/tests: project별 테스트와 expected output.
- pintos/utils: pintos, pintos-mkdisk, backtrace 같은 실행

공식 매뉴얼은 threads/를 project 1의 시작점으로 보고, userprog/, vm
확장된다고 설명한다. 이 repo도 같은 구조다.

개발 환경 활성화

터미널에서 Pintos root 기준으로 다음을 실행한다.

```
cd pintos
source ./activate
```

이 명령은 Pintos utility path를 잡아준다. pintos, pintos-mkdi
쓰려면 환경 활성화가 필요하다.

Project 1 빌드

```
cd pintos/threads
make
```

빌드가 성공하면 pintos/threads/build가 생긴다. 중요한 산출물은 다음이다.

- kernel.o: debug symbol이 있는 kernel object. backtrace와
- kernel.bin: 실제로 loader가 메모리에 올리는 kernel image.
- loader.bin: BIOS가 처음 읽는 512 byte loader image.
- subdirectory의 .o, .d: source별 object와 dependency.

테스트 실행

가장 단순한 실행 예:

```
cd pintos/threads/build
pintos -- run alarm-single
```

테스트 모음 실행:

```
cd pintos/threads
make check
```

특정 테스트만 돌리고 싶다면 보통 build directory에서 pintos -- run T
사용한다. 테스트 harness가 제공하는 명령은 프로젝트/환경에 따라 조금 다를 수 있으니
pintos/tests/threads/Make.tests, pintos/tests/tests.

실패 로그를 읽는 법

Pintos 테스트 실패는 대체로 다음 중 하나다.

- PANIC: 커널이 명시적으로 죽었다. assertion 또는 치명적인 상태.
- ASSERT failure: 함수의 전제조건이 깨졌다. 가장 좋은 힌트다.
- timeout: deadlock, busy wait, interrupt 상태 문제, threa
- output mismatch: 기능은 돌아가지만 순서, priority, exit messag
- page fault: user pointer 검증 실패, stack setup 오류, page
  크다.

실패 로그를 보면 먼저 "마지막으로 출력된 테스트 메시지"와 "panic/backtrace의
함수"를 잡는다. C library나 assembly frame보다, 내가 수정한 함수와 가까
function을 찾는 것이 빠르다.

3. Pintos 커널의 큰 흐름

부팅과 초기화 흐름

Pintos는 실제 PC처럼 boot loader에서 시작하지만, 공부할 때는 다음 흐름만 먼

1. loader assembly가 kernel image를 메모리에 올린다.
2. kernel entry가 C 초기화로 넘어온다.
3. threads/init.c의 main()이 subsystem을 초기화한다.
4. thread_init()이 현재 실행 중인 코드를 initial thread로 등록한다.
5. thread_start()가 idle thread를 만들고 interrupt를 켠다.
6. timer interrupt가 주기적으로 들어오며 preemption이 시작된다.
7. project 2 이후에는 process_create_initd()로 첫 user proces

Project 1에서 중요한 지점은 thread_start() 이후 interrupt가 켜지고 scheduling에 영향을 주기 시작한다는 것이다. 그 전까지는 cooperative한 초기 뒤부터는 언제든 현재 thread가 빼앗길 수 있다.

스레드 상태

struct thread의 상태는 네 가지다.

- THREAD_RUNNING: 현재 CPU에서 실행 중. 한 순간에 하나만 존재한다.
- THREAD_READY: 실행 가능하지만 아직 선택되지 않음. ready_list에 들어간다.
- THREAD_BLOCKED: 어떤 이벤트를 기다리는 중. ready list에 있으면 안 된다
- THREAD_DYING: 종료 중. scheduler가 안전한 시점에 정리한다.

상태 전이는 대략 다음과 같다.

```
thread create()
  -> init thread(): THREAD BLOCKED
  -> thread unblock(): THREAD READY
  -> schedule ??: THREAD RUNNING

thread vield()
  RUNNING -> READY -> schedule

thread block()
  RUNNING -> BLOCKED -> schedule

thread unblock(t)
  BLOCKED -> READY

thread exit()
  RUNNING -> DYING -> schedule
```

thread_block()은 저수준 함수다. 현재 thread를 block하고 바로 sched interrupt off 상태에서만 호출해야 한다. 그냥 편하다고 아무 데서나 호출하면 ass 터진다.

Timer interrupt와 preemption

pintos/devices/timer.c의 timer_interrupt()는 매 tick마다

핵심 작업:

- global ticks 증가.
- thread_tick() 호출.
- sleep list 같은 timer 기반 wait queue가 있다면 여기서 깨울 thread

thread_tick()은 현재 thread의 통계를 갱신하고, time slice가 끝나면 intr_yield_on_return()을 호출한다. interrupt handler 안에서 것이 아니라, interrupt에서 돌아갈 때 yield하도록 예약하는 점이 중요하다.

Scheduling의 기본 구조

thread.c의 기본 scheduler는 ready_list 앞에서 하나를 꺼내 실행한다.

```
static struct thread *
next_thread_to_run (void) {
  if (list_empty (&ready_list))
    return idle_thread;
  else
    return list_entry (list_pop_front (&ready_list), struct thread, elem);
}
```

기본 코드는 FIFO round-robin에 가깝다. priority scheduling을 구 priority 순서로 유지하거나, 꺼낼 때 가장 높은 priority를 찾아야 한다. 보통은 정렬을 유지하는 쪽이 테스트와 사고방식 모두 단순하다.

Interrupt off가 동기화인 이유

Pintos는 preemptible kernel이다. 즉 커널 코드 실행 중에도 timer i thread로 바뀔 수 있다. 따라서 shared state를 만질 때 동기화가 필요하다.

interrupt를 끄면 timer interrupt로 인한 preemption이 멈춘다. 그 waiters, sleep list 같은 scheduler 자료구조를 아주 짧게 수정할 때는 사용한다.

하지만 interrupt off는 무거운 도구다.

- 오래 끄면 timer와 device 반응성이 나빠진다.
- interrupt handler에서는 sleep할 수 없으므로 lock을 acquire하면 안
- 일반적인 긴 critical section은 lock/semaphore를 쓰는 것이 맞다.

정리하면, thread와 interrupt handler가 공유하는 scheduler 핵심 자 off로 보호하고, 일반 kernel thread끼리 공유하는 파일 시스템/프로세스 자료구조는 보호한다.

4. 꼭 알아야 할 C/OS 배경지식

포인터와 container_of 패턴

Pintos의 list_entry()와 hash_entry()는 Linux kernel의 co
노드가 데이터를 가리키는 것이 아니라, 데이터 구조체 안에 리스트 노드가 들어 있다.

```
struct thread {
  ...
  struct list elem elem;
  ...
};
```

리스트에는 struct thread *가 아니라 &t->elem을 넣는다. 나중에 struct
list_entry(e, struct thread, elem)로 원래 thread를 복원한다.

이 방식의 장점은 동적 할당 없이 리스트를 만들 수 있다는 것이다. 단점은 type check
없어서 실수하면 조용히 memory corruption이 난다는 것이다.

커널 stack 크기 제한

Pintos의 각 thread는 4KB page 하나를 받는다. 그 page의 아래쪽에는 st
위쪽은 kernel stack으로 쓴다. 그래서 struct thread를 너무 키우거나, k
local array를 만들면 stack overflow가 난다.

특히 다음을 조심한다.

- struct thread에 큰 배열을 넣지 않는다.
- 함수 local variable로 큰 buffer를 만들지 않는다.
- 깊은 recursion을 피한다.
- magic 필드는 struct thread 끝에 둔다. overflow 감지에 쓰인다.

ASSERT는 적이 아니라 지도다

Pintos의 assertion은 "이 함수는 이렇게 호출되어야 한다"라는 문서다. 예를 들어
thread_block()의 assertion이 실패하면 thread_block()을 고치는
interrupt off 조건을 지켰는지 봐야 한다.

자주 보는 assertion 의미:

- ASSERT (!intr_context ()): interrupt handler 안에서 호출하
- ASSERT (intr_get_level () == INTR_OFF): scheduler 자료
  꺼야 한다.
- ASSERT (is_thread (t)): thread pointer가 깨졌거나 stack o
- ASSERT (t->status == THREAD_BLOCKED): thread_unblock

x86-64 register calling convention

Project 2부터 사용자 프로그램과 시스템콜 때문에 register를 직접 다룬다. 일반
정수/포인터 인자는 보통 다음 순서의 register로 전달된다.

- 1번째: rdi

- 2번째: rsi
- 3번째: rdx
- 4번째: rcx
- 5번째: r8
- 6번째: r9

단, x86-64 syscall instruction에서는 4번째 인자가 rcx가 아니라 r1
intr_frame에는 R.rax, R.rdi, R.rsi, R.rdx, R.r10, R.r8

시스템콜 번호는 R.rax에 들어오고, return value도 R.rax에 넣어 돌려준다.

User address와 kernel address

Pintos는 user virtual address와 kernel virtual address
포인터를 커널이 믿고 바로 dereference하면 안 된다.

검사해야 할 것:

- 포인터가 NULL인지.
- user address range 안인지.
- 해당 page가 실제 mapping되어 있는지.
- buffer가 여러 page를 걸칠 때 모든 page가 유효한지.
- write buffer라면 writable mapping인지.

Project 2에서는 page table을 직접 확인해서 검사하고, Project 3에서는
loading까지 고려해야 한다.

5. Pintos list 사용법

Pintos list의 정체

pintos/include/lib/kernel/list.h의 list는 doubly linke list처럼 node가 data pointer를 들고 있지 않다. 대신 리스트에 들어갈 구조체 list_elem 멤버를 가진다.

예시:

```
struct foo {
  int value;
  struct list_elem elem;
};

struct list foo_list;
list_init (&foo_list);
```

삽입:

```
struct foo f;
f.value = 10;
list_push_back (&foo_list, &f.elem);
```

순회:

```
struct list_elem *e;
for (e = list_begin (&foo_list); e != list_end (&foo_list); e = list_next (e)) {
  struct foo *f = list_entry (e, struct foo, elem);
  /* use f */
}
```

핵심은 list_entry(e, struct foo, elem)이다. 이 매크로는 elem의 구조체인 struct foo의 시작 주소를 계산한다.

반드시 기억할 규칙

- struct list는 list header다. list_init(&list)로 초기화한다.
- struct list_elem은 실제 element link다. 보통 자료구조 안에 embed
- 한 struct list_elem은 동시에 한 list에만 들어갈 수 있다.
- 같은 object가 동시에 여러 list에 들어가야 하면 list별로 struct list_e
- 빈 list에서 list_front(), list_back(), list_pop_front()
- 순회 중 remove할 때는 list_remove()의 return value를 다음 elem

잘못된 remove pattern:

```
for (e = list_begin (&list); e != list_end (&list); e = list_next (e)) {
  if (condition)
    list_remove (e);
}
```

올바른 remove pattern:

```
e = list_begin (&list);
while (e != list_end (&list)) {
  struct foo *f = list_entry (e, struct foo, elem);
  if (condition) {
    e = list_remove (e);
```

```
  } else {
    e = list_next (e);
  }
}
```

정렬 삽입

priority scheduling이나 alarm clock에서는 ordered list가 편
comparator를 받아 알맞은 위치에 삽입한다.

```
static bool
foo_less (const struct list_elem *a,
          const struct list_elem *b,
          void *aux UNUSED) {
  const struct foo *fa = list_entry (a, struct foo, elem);
  const struct foo *fb = list_entry (b, struct foo, elem);
  return fa->value < fb->value;
}

list_insert_ordered (&foo_list, &f.elem, foo_less, NULL);
```

priority가 큰 thread를 앞에 두려면 comparator는 보통 a_priority
만든다. sleep list를 빠른 wakeup tick 순서로 두려면 a_wakeup_tic
반환한다.

timer sleep에서 list를 쓸 때

Project 1 alarm clock의 sleep list는 보통 timer.c에 둔다.

```
static struct list sleep_list;
```

timer_init() 또는 별도 초기화 지점에서:

```
list_init (&sleep_list);
```

struct thread에는 wakeup tick 필드를 추가한다.

```
int64_t wakeup_tick;
```

잠들 때는 현재 thread의 elem을 sleep list에 넣고 block한다. 이 thr
ready list에 없으므로 기본 elem을 sleep list에 재사용할 수 있다. 하지만
waiters에도 들어갈 수 있는 설계를 만들면 안 된다. 하나의 elem은 한 리스트만 가능

extern struct list time_list가 안 되는 이유

현재 pintos/threads/thread.c에는 실습 중 추가된 것으로 보이는 static
time_list;가 있다. static 전역 변수는 그 .c 파일 내부에서만 보인다. 따라서
다음처럼 선언해도 linker가 찾을 수 없다.

```
extern struct list time_list;
```

해결 방향은 둘 중 하나다.

- sleep list를 실제로 사용하는 timer.c 안에 static struct list s
- thread module이 sleep queue를 소유하도록 설계하고, 외부에는 함수 inte

학습 단계에서는 timer.c가 timer sleep의 책임을 갖게 두는 쪽이 이해하기 쉽다.

6. Project 1: Threads

Project 1의 목표

Project 1은 "스레드를 OS답게 다루기"가 목표다. 세부 과제는 보통 다음이다.

- Alarm clock: busy waiting 없이 sleep 구현.
- Priority scheduling: 높은 priority thread가 먼저 실행되게 만들기
- Priority donation: lock 때문에 생기는 priority inversion 완
- Advanced scheduler: MLFQS와 fixed-point arithmetic.

이 과제는 뒤 project에서 직접 재사용되지 않는 부분도 있지만, Pintos 전체의 감각 특히 interrupt off, ready list, semaphore waiters, str 나온다.

6.1 Alarm Clock

요구사항

timer_sleep(ticks)는 현재 thread를 최소 ticks timer tick 동 구현은 반복문에서 thread_yield()를 호출하며 시간을 확인하는 busy wait 방식 CPU를 계속 쓰므로 테스트가 싫어한다.

올바른 방향:

- 현재 thread에 wakeup tick을 기록한다.
- sleep list에 넣는다.
- current thread를 block한다.
- timer interrupt마다 sleep list를 확인한다.
- wakeup tick이 지난 thread를 unblock한다.

핵심 파일

- pintos/devices/timer.c: timer_sleep(), timer_interru
- pintos/include/threads/thread.h: struct thread에 slee
- pintos/threads/thread.c: thread_block(), thread_unbl
- pintos/tests/threads/alarm-*.c: 기대 동작 확인.

구현 사고 순서

1. struct thread에 int64_t wakeup_tick; 추가.
2. timer.c에 static struct list sleep_list; 추가.
3. timer 초기화 시 list_init(&sleep_list).
4. timer_sleep(ticks)에서 ticks <= 0이면 바로 return.
5. interrupt를 끄고 현재 thread의 wakeup_tick = timer_ticks()
6. sleep list에 삽입.
7. thread_block().
8. 이전 interrupt level 복구.
9. timer_interrupt()에서 ticks 증가 후 sleep list를 순회하며 깰 th

interrupt 관련 주의점

thread_block()은 interrupt off 상태에서 호출해야 한다. 따라서 다음 형

```
enum intr_level old_level = intr_disable ();
/* update current thread and sleep list */
thread_block ();
intr_set_level (old_level);
```

timer_interrupt()는 interrupt context다. 여기서는 sleep할 수 block될 수 있는 일을 하면 안 된다. 리스트를 순회하고 thread_unblock()처럼 context에서 허용되는 짧은 작업만 해야 한다.

정렬 list를 쓰는 이유

sleep list를 wakeup tick 순서로 정렬해 두면 timer interrupt에서 시간이 안 된 thread를 만나는 순간 멈출 수 있다.

정렬하지 않으면 매 tick마다 전체 sleep list를 훑어야 한다. 테스트 규모에서는 통 있어도 설계상 비효율적이다.

정렬 sleep list의 불변식:

- list 앞쪽일수록 wakeup tick이 작다.
- timer_interrupt()는 list front부터 깨운다.
- front가 아직 깰 시간이 아니면 뒤도 모두 아직이다.

alarm 테스트 해석

- alarm-single: thread 하나가 sleep 후 깨어나는 기본 동작.
- alarm-multiple: 여러 thread가 서로 다른 tick에 깨어나는지.
- alarm-simultaneous: 같은 tick에 여러 thread가 깨어나는지.
- alarm-zero, alarm-negative: 0 이하 sleep이 이상하게 block되지
- alarm-priority: priority scheduling과 결합했을 때 깨어난 thre

6.2 Priority Scheduling

요구사항

priority가 높은 thread가 낮은 thread보다 먼저 CPU를 받아야 한다. 새 t 들어왔는데 현재 thread보다 priority가 높다면 현재 thread는 즉시 yield해

또한 semaphore, lock, condition variable에서 waiters를 깨울 먼저 깨워야 한다.

핵심 변경 지점

- thread_unblock(): ready list에 priority 순서로 삽입.
- thread_yield(): 현재 thread를 ready list에 다시 넣을 때 prior
- next_thread_to_run(): ready list front가 highest prio
- thread_create(): 새 thread 생성 후 preemption 필요 여부 확인.
- thread_set_priority(): priority를 낮춘 뒤 더 높은 ready thr
- sema_down(): waiters list 삽입 순서.
- sema_up(): highest priority thread unblock.
- cond_signal(): condition waiter 중 highest priority를

ready list 불변식

추천 불변식:

- ready_list는 effective priority 내림차순이다.
- list_front(&ready_list)는 현재 실행 가능한 thread 중 가장 높은 pr
- thread_unblock()과 thread_yield()는 항상 ordered insert를
- priority가 바뀐 thread가 ready list 안에 있다면 list 순서를 다시 맞

즉시 선점

선점이 필요한 상황:

- 새 thread가 unblock되었고 current보다 priority가 높다.
- current thread가 자신의 priority를 낮췄고 ready list front가
- lock release 후 donation이 사라져 current effective prior

interrupt context에서는 바로 thread_yield()를 호출할 수 없다. 이
intr_yield_on_return()을 써야 한다. 일반 thread context에서는

6.3 Priority Donation

priority inversion 문제

세 thread가 있다고 하자.

- H: high priority.
- M: medium priority.
- L: low priority.

L이 lock A를 들고 있다. H가 lock A를 얻으려다 block된다. M은 lock이
상태다. priority scheduling만 있으면 M이 L보다 우선 실행되어 L이 lock
줄어든다. 결과적으로 H가 M 때문에 간접적으로 굶는다. 이것이 priority inversi

해결은 H의 priority를 lock holder인 L에게 임시로 빌려주는 것이다. L이 빨
release하면 H가 깨어난다.

필요한 필드 예시

구현 방식은 여러 가지지만 보통 struct thread에 다음 개념이 필요하다.

- base priority: 사용자가 설정한 원래 priority.
- effective priority: donation까지 반영한 현재 scheduling pri
- waiting lock: 현재 이 thread가 기다리는 lock.
- donations list: 이 thread에게 priority를 donation한 threa
- donation elem: donations list에 들어가기 위한 별도 list eleme

중요한 점은 struct list_elem elem 하나를 ready list와 semapho
것이다. donation list에 같은 elem을 또 넣으면 안 된다. 필요하면 별도의 st
donation_elem;을 추가한다.

lock acquire 흐름

lock acquire에서 donation이 일어나는 지점:

1. current thread가 lock을 얻으려 한다.
2. lock holder가 이미 있다.
3. current의 waiting_lock을 해당 lock으로 설정한다.

4. holder에게 current priority를 donation한다.
5. holder가 또 다른 lock을 기다리고 있으면 donation을 chain으로 전파한다.
6. sema_down()으로 실제 block한다.
7. 깨어나 lock을 얻으면 waiting_lock = NULL, lock->holder = cu

중첩 donation은 무한히 따라가면 위험하므로 공식 요구사항도 합리적 depth 제한을 허
흔히 8 정도를 사용한다.

lock release 흐름

lock release에서 해야 할 일:

1. current가 해당 lock holder인지 확인.
2. 이 lock 때문에 current에게 들어온 donation을 제거한다.
3. current effective priority를 다시 계산한다.
4. lock holder를 NULL로 만든다.
5. semaphore up으로 waiter를 깨운다.
6. 더 높은 priority ready thread가 있으면 yield한다.

donation 제거가 어려운 이유는 "어떤 donation이 어떤 lock 때문에 생겼는가"
때문이다. donation thread의 waiting_lock을 보면 필터링할 수 있다.

priority 관련 테스트

- priority-fifo: 같은 priority에서는 FIFO 성질이 유지되는지.
- priority-preempt: 높은 priority thread가 생기면 즉시 선점되는지.
- priority-change: priority 변경 후 yield 여부.
- priority-sema: semaphore waiters가 priority 순서로 깨어나는지
- priority-condvar: condition variable waiter 선택이 prio
- priority-donate-one: 단일 donation.
- priority-donate-multiple: 여러 thread가 하나에게 donation.
- priority-donate-nest: donation chain.
- priority-donate-sema: lock과 semaphore 대기 상황이 섞일 때.
- priority-donate-lower: donation 중 base priority를 낮춰도 유지되는지.

6.4 Advanced Scheduler, MLFQS

개념

MLFQS는 priority를 직접 donation하지 않고, nice, recent_cpu, thread_mlfqs가 true일 때 priority donation은 보통 사용하지 않는다

공식 공식은 다음 형태다.

```
priority = PRI_MAX - (recent_cpu / 4) - (nice * 2)
recent_cpu = (2 * load_avg) / (2 * load_avg + 1) * recent_cpu + nice
load_avg = (59 / 60) * load_avg + (1 / 60) * ready_threads
```

Pintos kernel에는 floating point를 쓰면 안 된다. 그래서 fixed-p
구현한다.

fixed point 기본

보통 17.14 fixed point를 사용한다.

```
#define F (1 << 14)

int int_to_fp (int n) { return n * F; }
int fp_to_int_zero (int x) { return x / F; }
int fp_to_int_nearest (int x) {
  return x >= 0 ? (x + F / 2) / F : (x - F / 2) / F;
}
int add_fp (int x, int y) { return x + y; }
int sub_fp (int x, int y) { return x - y; }
int mul_fp (int x, int y) { return ((int64_t) x) * y / F; }
int div_fp (int x, int y) { return ((int64_t) x) * F / y; }
```

곱셈과 나눗셈에서 int64_t로 올리지 않으면 overflow가 쉽게 난다.

업데이트 타이밍

일반적으로:

- 매 tick: running thread가 idle이 아니면 recent_cpu++.
- 매 초, 즉 timer_ticks() % TIMER_FREQ == 0: load_avg와 모든
- 매 4 tick: 모든 thread priority 재계산.

모든 thread를 순회하려면 all threads list가 필요할 수 있다. 기본 Pint
있으므로 구현 설계에 따라 all_list를 유지한다.

7. Project 2: User Programs

Project 2의 큰 목표

Project 2부터 Pintos는 kernel thread만 돌리는 것이 아니라 user p
실행한다. 이제 kernel은 신뢰할 수 없는 user code를 상대해야 한다.

핵심 목표:

- command line argument를 user stack/register에 올바르게 전달.
- system call handler 구현.
- user pointer 검증.
- process wait/exit/fork/exec lifetime 관리.
- file descriptor table 구현.
- executable file write deny 처리.

Project 2의 핵심 기준은 "어떤 user program도 kernel을 crash시키면
program은 죽어도 된다. kernel은 죽으면 안 된다.

7.1 process lifecycle

주요 함수:

- process_create_initd(file_name): 첫 user process 생성.
- initd(f_name): initial user process thread entry.
- process_exec(f_name): 현재 process image를 새 executable
- load(file_name, if_): ELF 파일을 열고 segment와 stack을 구성.
- setup_stack(if_): user stack page를 만든다.
- process_wait(child_tid): child 종료 대기와 exit status 회수
- process_exit(): process 자원 정리.

현재 repo의 process.c에는 TODO가 많다. 특히 process_wait(), pr
process_fork(), duplicate_pte()는 project 2의 핵심 구현 지점

7.2 ELF loading

load()는 executable file을 열고 ELF header와 program head
segment마다 file data를 user virtual address에 map한다.

Project 2, VM 비활성 상태에서는 load_segment()가 즉시 page를 할당한

```
file offset -> kernel page? read
?? ?? zero fill
user virtual page -> kernel physical page mapping
```

Project 3 이후 VM 활성 상태에서는 lazy loading으로 바뀐다. 즉 load_
바로 읽지 않고, supplemental page table에 "나중에 fault가 나면 이
읽어라"라는 정보를 넣는다.

7.3 Argument Passing

KAIST x86-64 Pintos에서 user program entry는 _start(arg

lib/user/entry.c에 있고, 결국 main(argc, argv)를 호출한 뒤 exi

해야 할 일:

1. command line을 token으로 나눈다.
2. program name은 executable open에 사용한다.
3. argument string들을 user stack 위쪽부터 복사한다.
4. 각 string의 user virtual address를 기록한다.
5. argv[] pointer array를 stack에 배치한다.
6. stack alignment를 맞춘다.
7. fake return address를 넣는다.
8. if_->R.rdi = argc, if_->R.rsi = argv_address를 설정한다.

주의할 점:

- strtok_r()처럼 reentrant tokenizer를 사용한다.
- command line 원본 page는 load 후 free되어야 한다.
- stack pointer는 user virtual address 기준이다.
- 문자열은 끝의 \0까지 복사해야 한다.
- alignment가 틀리면 단순 args 테스트도 실패할 수 있다.

스택 배치 개념:

```
high address: USER STACK
  "arg strings..."
  padding for alignment
  NULL sentinel
  argv[n-1]
  ...
  argv[0]
  fake return address
low address
```

단, x86-64에서는 argc/argv 자체는 register로 전달된다. stack에는 a
pointer array를 준비한다.

7.4 System Call

pintos/userprog/syscall.c의 syscall_handler(struct in
현재 skeleton은 system call!을 출력하고 thread를 종료한다.

시스템콜 번호:

- SYS_HALT
- SYS_EXIT
- SYS_FORK
- SYS_EXEC
- SYS_WAIT
- SYS_CREATE
- SYS_REMOVE
- SYS_OPEN
- SYS_FILESIZE
- SYS_READ
- SYS_WRITE
- SYS_SEEK
- SYS_TELL

- SYS_CLOSE
- Project 3 이후 SYS_MMAP, SYS_MUNMAP
- Project 4 이후 directory 관련 syscall

인자 register:

```
number: rax
arg1:   rdi
arg2:   rsi
arg3:   rdx
arg4:   r10
arg5:   r8
arg6:   r9
return: rax
```

handler는 보통 다음 흐름이다.

```
void
syscall_handler (struct intr_frame *f) {
  switch (f->R.rax) {
    case SYS_EXIT:
      sys_exit ((int) f->R.rdi);
      break;
    case SYS_WRITE:
      f->R.rax = sys_write ((int) f->R.rdi,
                            (const void *) f->R.rsi,
                            (unsigned) f->R.rdx);
      break;
    ...
  }
}
```

이 코드는 개념 예시다. 실제 구현에서는 user pointer 검증과 lock 처리, fd
반드시 들어간다.

7.5 User Pointer 검증

시스템콜 인자로 들어오는 포인터는 모두 user가 준 것이다. user는 악의적일 수 있다고
가정한다.

검증해야 할 포인터 종류:

- C string: exec(const char *cmd_line), open(const cha
- read buffer: kernel/file에서 user memory로 써야 하므로 writa
- write buffer: user memory에서 kernel/file로 읽어야 하므로 rea
- pointer array: argv 등.

Project 2에서 단순 접근 방법:

- is_user_vaddr(ptr)로 user range 확인.
- pml4_get_page(thread_current()->pml4, ptr)로 mapping
- buffer가 page boundary를 넘으면 각 page마다 확인.

Project 3에서는 lazy loading 때문에, page fault를 통해 page를
pointer validation 설계를 VM과 충돌하지 않게 만들어야 한다.

잘못된 포인터를 받았을 때의 목표는 단순하다. user process는 종료될 수 있지만 ke
panic하면 안 된다.

7.6 File Descriptor Table

각 process는 독립적인 file descriptor table을 가진다. fd 0은 st
open()은 보통 2 이상의 fd를 반환한다.

구현 선택지:

- 고정 크기 array.
- list.
- hash map.

처음에는 array가 쉽다. 하지만 fork, dup2, 많은 fd 테스트를 생각하면 확장 가
편할 수 있다.

fd table entry가 가져야 할 정보:

- fd number.
- struct file *.
- directory fd인지 file fd인지 project 4 대비 정보.
- closed 여부.

read()와 write()의 특별 처리:

- read(0, buffer, size): keyboard input, input_getc().
- write(1, buffer, size): console output, putbuf() 사용.
- stdin에 write하거나 stdout에서 read하는 경우는 실패 처리.

파일 시스템은 project 2 시점에 내부 synchronization이 충분하지 않다. 공
system code를 critical section처럼 보호하라고 한다. 따라서 syscal
global file system lock을 두는 방식이 일반적이다.

7.7 wait와 exit

wait(pid)는 단순히 thread가 죽을 때까지 기다리는 함수가 아니다. child re
status 수거 semantics가 있다.

필수 조건:

- direct child만 wait 가능.
- 같은 child를 두 번 wait하면 두 번째는 실패.
- child가 이미 죽었어도 parent가 아직 wait하지 않았다면 status를 받을 수 있
- parent가 먼저 죽어도 child resource가 누수되면 안 된다.
- child가 load에 실패하면 parent의 exec 또는 fork가 실패를 알아야 한다.

보통 child info 구조체를 둔다.

```
struct child_info {
  tid_t tid;
  int exit_status;
  bool waited;
  bool exited;
  struct semaphore wait_sema;
  struct list_elem elem;
};
```

그리고 parent의 struct thread에 children list를 둔다. child가
child_info에 status를 기록하고 semaphore up을 한다. parent는 w

찾고 semaphore down으로 기다린다.

7.8 fork와 exec

fork는 현재 process를 복제한다. parent와 child의 차이:

- parent의 return value는 child pid.
- child의 return value는 0.
- child는 parent의 address space, file descriptors, 필요한
- parent는 child 복제가 성공했는지 알기 전까지 return하면 안 된다.

exec는 현재 process image를 새 executable로 교체한다. file des
돌아오지 않는다. 실패하면 보통 exit status -1로 종료된다.

fork는 구현량이 크다. Project 2에서는 page table을 복제하고, Projec
page table과 lazy page 정보를 복제해야 한다.

7.9 Denying Writes to Executables

실행 중인 executable file에 write가 허용되면 running code가 바뀌는
Pintos tests는 실행 파일에 대한 write deny를 확인한다.

기본 방향:

- process가 executable을 열 때 file_deny_write(file) 호출.
- struct thread에 executable file pointer 저장.
- process exit 또는 exec cleanup 때 file_allow_write() 후

load 실패 경로에서도 file close와 deny 상태 정리를 놓치면 안 된다.

8. Project 3: Virtual Memor

Project 3의 큰 목표

Project 3은 "page fault를 정상적인 실행 경로로 받아들이는 것"이 핵심이다. 2에서는 executable segment를 즉시 메모리에 올렸다. Project 3에서는 필 frame에 올린다.

핵심 구성:

- supplemental page table, SPT.
- page object와 page operation table.
- frame table과 eviction.
- anonymous page와 swap.
- file-backed page와 mmap.
- stack growth.
- copy-on-write extra.

8.1 page, frame, PTE

용어를 분리해야 한다.

- page: user virtual page. 예: user virtual address 0x4
- frame: physical memory page. kernel virtual address
- PTE/page table entry: virtual page와 physical frame의
- supplemental page table: page fault를 처리하기 위한 추가 meta

struct page는 virtual memory object다. struct frame은 p page->frame과 frame->page는 서로 back reference를 갖는다.

8.2 struct page_operations

pintos/include/vm/vm.h는 C에서 다형성을 흉내 낸다.

```
struct page_operations {
  bool (*swap_in) (struct page *, void *);
  bool (*swap_out) (struct page *);
  void (*destroy) (struct page *);
  enum vm_type type;
};
```

page type마다 swap in/out/destroy 방식이 다르다.

- VM_UNINIT: 아직 실제 내용이 초기화되지 않은 lazy page.
- VM_ANON: file backing이 없는 anonymous page. swap disk가
- VM_FILE: file-backed page. mmap 또는 lazy loaded execu
- VM_PAGE_CACHE: project 4 page cache에 사용 가능.

swap_in(page, kva)는 실제 page 내용을 frame에 채운다. swap_out backing store로 내보낸다. destroy(page)는 page type별 자원을 정

8.3 Supplemental Page Table 설계

SPT는 struct supplemental_page_table에 구현한다. 공식 skelet
가장 흔한 설계는 hash table이다.

왜 hash인가:

- key는 page-aligned virtual address.
- page fault 때 addr에 해당하는 page를 빠르게 찾아야 한다.
- Pintos에 include/lib/kernel/hash.h가 이미 있다.

SPT hash element를 넣으려면 struct page에 struct hash_elem
둔다. 대체로 struct page에 직접 넣는 방식이 간단하다.

필수 함수:

- supplemental_page_table_init(spt): hash init.
- spt_find_page(spt, va): pg_round_down(va)로 page 찾기.
- spt_insert_page(spt, page): 중복 없이 삽입.
- spt_remove_page(spt, page): 제거 후 dealloc.
- supplemental_page_table_copy(dst, src): fork에서 복제.
- supplemental_page_table_kill(spt): process exit 때 모든

SPT 불변식:

- 한 user virtual page address에는 page object가 최대 하나.
- page key는 page-aligned address.
- page가 SPT에 있으면 그 page의 lifetime은 SPT cleanup에서 관리된다.

8.4 Lazy Loading

lazy loading은 "지금 당장 파일을 읽지 않고, page fault가 날 때 읽는다"

ELF loading에서 해야 할 일:

1. segment별 page 정보를 계산한다.
2. page마다 read bytes, zero bytes, file offset, writable
3. vm_alloc_page_with_initializer()로 uninit page를 SPT에
4. 실제 user code가 해당 address에 접근하면 page fault 발생.
5. vm_try_handle_fault()가 SPT에서 page를 찾는다.
6. vm_claim_page()가 frame을 얻고 PTE mapping을 만든다.
7. uninit page의 initializer가 file에서 내용을 읽고 zero fill한다.

lazy loading에서 자주 하는 실수:

- aux가 stack local variable을 가리키게 만든다. fault 시점에는 이미 사
- file pointer lifetime을 관리하지 않는다.
- read bytes/zero bytes 계산을 page마다 잘못 넘긴다.
- writable flag를 PTE mapping에 반영하지 않는다.

aux는 heap allocation하거나 page 구조체 안에 필요한 정보를 복사해야 한다.

8.5 Page Fault 처리

userprog/exception.c의 page fault handler는 결국 vm_try_
함수는 fault가 처리 가능한지 판단해야 한다.

처리 가능한 경우:

- not-present page이고 SPT에 해당 page가 있다.
- lazy load 대상이다.
- swapped out page다.
- file-backed page다.
- stack growth로 인정되는 access다.

처리하면 안 되는 경우:

- kernel virtual address 접근.
- NULL 또는 page 0 근처 invalid access.
- write access인데 page가 read-only.
- stack growth heuristic에 맞지 않는 큰 jump.
- present page protection violation인데 COW 등으로 처리할 계획이

vm_try_handle_fault()는 invalid access를 true로 처리하면 안 false를 반환해서 user process가 종료되게 해야 한다.

8.6 Frame Table과 Eviction

physical memory는 제한되어 있다. palloc_get_page(PAL_USER)가 필요하다.

frame table이 해야 할 일:

- 현재 사용 중인 frame 목록 유지.
- eviction victim 선택.
- victim page의 swap_out() 호출.
- page table mapping 제거.
- frame을 새 page에 재사용.

victim policy는 자유지만 보통 clock algorithm을 쓴다. PTE acce 사용 여부를 볼 수 있다.

clock algorithm 개념:

```
frame list? ???? ??.
accessed bit? 1?? 0?? ??? ? ? ???.
accessed bit? 0?? victim?? ????.
```

eviction 중에는 frame table lock이 필요하다. 동시에 여러 thread가 evict하면 같은 frame을 두 page가 쓰는 사고가 난다.

8.7 Anonymous Page와 Swap

anonymous page는 특정 file이 backing store가 아닌 page다. st 여기에 해당한다. 메모리에서 쫓겨날 때는 swap disk에 내용을 저장해야 한다.

구현 요소:

- swap disk pointer.
- swap slot bitmap.
- anon page 안의 swap slot index.
- swap table lock.

swap out:

1. free swap slot을 찾는다.

2. frame의 4KB 내용을 disk sector 여러 개에 기록한다.
3. page에 slot index를 저장한다.
4. PTE mapping을 제거한다.
5. frame과 page 연결을 끊는다.

swap in:

1. page의 slot index를 확인한다.
2. disk에서 frame으로 4KB를 읽는다.
3. slot을 free로 표시한다.
4. page의 slot index를 invalid로 바꾼다.

swap slot이 없으면 kernel panic을 허용하는 설계도 가능하다. 하지만 slot corruption은 허용하면 안 된다.

8.8 Stack Growth

Project 2에서는 stack page가 하나였다. Project 3에서는 stack이 아 동적으로 할당한다.

heuristic:

- fault address가 user stack 근처여야 한다.
- fault address가 현재 user rsp보다 너무 아래면 안 된다.
- x86-64 push는 rsp보다 8 byte 아래 fault를 낼 수 있으므로 이 범위를 허
- 최대 stack 크기는 보통 1MB로 제한한다.

문제는 kernel mode page fault에서 intr_frame.rsp가 user rs 따라서 syscall 진입 시 user rsp를 struct thread에 저장해두는 설계가

stack growth flow:

```
page fault addr
  -> valid stack access?? ??
  -> addr? page boundary? round down
  -> USER_STACK?? ?? 1MB ???? ??
  -> anonymous page allocate
  -> claim
```

8.9 Memory Mapped Files

mmap은 file 내용을 process address space에 mapping한다. fil file에서 읽고, dirty하면 unmap 또는 eviction 때 file에 write b

mmap 검증 조건:

- addr가 page aligned인지.
- length가 0이 아닌지.
- fd가 valid file인지.
- offset이 page aligned인지.
- mapping range가 기존 page와 겹치지 않는지.
- NULL address 또는 kernel address가 아닌지.

munmap은 mapping 범위의 page들을 제거한다. dirty page는 file에 w

file-backed eviction:

- clean page라면 그냥 버릴 수 있다.

- dirty page라면 file offset에 write back한다.
- write back byte 수는 마지막 page에서 file length를 넘지 않게 계산한

8.10 Copy-on-Write 개념

COW는 extra 과제일 수 있다. 기본 아이디어는 fork 때 page 내용을 즉시 복사하
parent/child가 같은 frame을 read-only로 공유하다가 write fault

필요한 것:

- frame reference count.
- page/PTE writable 상태 조정.
- write-protection fault 처리.
- shared frame lock.

COW는 멋지지만 기본 VM이 안정된 뒤에 해야 한다. SPT, frame eviction,
상태에서 COW를 얹으면 디버깅 난도가 크게 올라간다.

9. Project 4: File System

Project 4의 큰 목표

Project 4는 Pintos의 단순 파일 시스템을 실제 파일 시스템답게 확장한다.

주요 목표:

- indexed and extensible files.
- subdirectories.
- current working directory.
- soft links.
- buffer cache.
- file system 내부 synchronization.

Project 2에서는 file system 전체를 global lock으로 감싸도 되지만, 세밀한 동기화가 요구된다.

9.1 기본 파일 시스템 구조

중요 파일:

- filesys/filesys.c: high-level create/open/remove.
- filesys/file.c: open file object, read/write/seek/te
- filesys/inode.c: inode metadata와 data block 접근.
- filesys/directory.c: directory entry 관리.
- filesys/free-map.c: free sector 관리.
- filesys/fat.c: FAT 기반 free/block 관리가 포함된 버전.
- filesys/page_cache.c: buffer cache extra 또는 확장 구현 위치

기본 계층:

```
syscall layer
  -> filesys/file API
  -> inode API
  -> disk sector read/write
```

file descriptor는 process별 handle이고, struct file은 열린 disk상의 파일 metadata를 대표한다.

9.2 Indexed and Extensible Files

기본 파일 시스템은 fixed-size file에 가깝다. 확장 과제에서는 file이 커질 때 block을 할당할 수 있어야 한다.

일반적인 indexed inode 설계:

- direct block pointers.
- single indirect block pointer.
- double indirect block pointer.

block lookup은 file offset을 sector index로 바꾼 뒤, direc 어디에 있는지 계산한다.

파일 확장 시 고려할 것:

- 새 sector를 free map에서 할당한다.
- 새 sector를 zero로 초기화한다.
- indirect block 자체가 필요하면 먼저 할당한다.
- 중간 hole이 생기면 읽을 때 zero를 반환해야 한다.
- 확장 중 실패하면 이미 할당한 sector를 rollback해야 한다.

atomicity도 중요하다. 두 thread가 동시에 같은 file을 확장하면 inode m
된다.

9.3 Subdirectories

directory도 결국 inode를 가진 file이다. directory file 안에는 n
연결하는 entry들이 들어 있다.

필요 기능:

- mkdir: directory inode 생성, .와 .. entry 설정.
- chdir: process의 current working directory 변경.
- readdir: directory entry 순회.
- isdir: fd가 directory인지 확인.
- inumber: inode number 반환.
- path parsing: absolute path와 relative path 처리.

path parsing은 Project 4에서 디버깅 시간을 많이 먹는다. 다음 질문을 항상

- /a/b/c와 a/b/c를 다르게 시작하는가?
- 중간 component가 file이면 실패하는가?
- 마지막 component가 없거나 빈 문자열이면 어떻게 처리하는가?
- root의 parent는 root로 처리하는가?
- current working directory가 삭제된 경우를 어떻게 처리하는가?

9.4 Soft Links

symlink는 "다른 path를 내용으로 가진 파일 같은 object"다. open/path
symlink를 만나면 target path로 다시 따라간다.

주의점:

- symlink loop 방지 depth limit이 필요하다.
- symlink target이 absolute path인지 relative path인지에 따라
  있다.
- symlink 자체를 삭제하는 것과 target을 삭제하는 것은 다르다.

9.5 Buffer Cache

buffer cache는 disk sector read/write를 메모리에 cache한다.
synchronization 구조 개선이다.

cache entry가 가질 정보:

- sector number.
- data buffer.
- valid bit.

- dirty bit.
- accessed bit.
- pin count 또는 in-use 상태.
- per-entry lock.

cache lookup flow:

```
sector ??
  -> cache hit?? entry ??
  -> miss?? victim ??
  -> dirty victim?? disk? write back
  -> requested sector? disk?? read
  -> entry ??
```

write-behind와 read-ahead는 추가 최적화다. 먼저 correctness가 우

9.6 File System Synchronization

Project 4 공식 요구는 global file system lock보다 세밀한 동시성을

동기화 방향:

- 서로 다른 cache block 작업은 독립적으로 진행 가능해야 한다.
- 같은 file에 대한 여러 read는 동시에 가능해야 한다.
- file extension은 atomic해야 한다.
- 서로 다른 directory 작업은 가능하면 독립적이어야 한다.
- 같은 directory entry set을 바꾸는 작업은 보호해야 한다.

처음부터 reader-writer lock을 크게 설계하기보다, inode lock, dire
lock의 책임을 명확히 하는 것이 좋다.

10. 디버깅과 테스트 읽는 법

printf 디버깅

Pintos에서 printf는 강력하지만 조심해서 써야 한다.

- interrupt handler 안에서 너무 많이 출력하면 timing이 바뀐다.
- scheduler switching 중간에는 출력이 위험할 수 있다.
- 테스트 expected output을 깨뜨릴 수 있다.
- user program output과 kernel debug output이 섞이면 gradin

디버깅 출력은 조건부로 짧게 넣고, 통과 전에는 제거한다.

ASSERT 추가하기

내 코드에 ASSERT를 추가하는 것은 좋다. 특히 불변식이 중요한 자료구조에 유용하다.

예:

- sleep list에 넣는 thread는 current이고 running 상태여야 한다.
- thread_unblock() 대상은 blocked 상태여야 한다.
- fd table lookup 결과가 NULL이면 file operation을 하지 않는다.
- SPT insert 전 같은 va가 없어야 한다.
- frame을 claim할 때 frame->page == NULL이어야 한다.

ASSERT는 증상을 빨리 폭발시켜준다. 조용한 memory corruption보다 훨씬 낫다

Backtrace

Pintos의 panic 로그에 주소가 나오면 utils/backtrace를 사용할 수 있다. symbol이 필요하다.

일반 패턴:

```
cd pintos/threads/build
backtrace kernel.o 0xADDRESS1 0xADDRESS2 ...
```

backtrace에서 가장 중요한 것은 "내가 수정한 코드와 가장 가까운 frame"이다.

GDB 기본 루틴

환경별 명령은 조금 다를 수 있지만 보통 흐름은 다음과 같다.

1. Pintos를 gdb 대기 모드로 실행.
2. 다른 terminal에서 gdb로 kernel.o attach.
3. breakpoint 설정.
4. continue.

볼 만한 breakpoint:

- timer_sleep
- timer_interrupt
- thread_block

- thread_unblock
- schedule
- syscall_handler
- page_fault
- vm_try_handle_fault
- process_exit

자주 쓰는 gdb 명령:

```
b timer_sleep
b syscall_handler
b page_fault
c
bt
p/x *f
p thread_current()->name
p thread_current()->status
```

테스트 파일 읽기

테스트는 최고의 명세다. 예를 들어 pintos/tests/threads/alarm-wait. 순서와 시간을 기대하는지 알 수 있다.

.ck 파일은 expected output pattern이다. C test와 .ck를 같이 읽 만들어야 하는지"가 분명해진다.

테스트 읽는 순서:

- C test가 어떤 helper를 쓰는지 본다.
- 몇 개 thread/process를 만드는지 본다.
- priority, sleep tick, fd, address 같은 입력값을 확인한다.
- .ck에서 기대 output 순서를 확인한다.
- timeout 가능성이 있는 wait 지점을 찾는다.

timeout 디버깅

timeout은 보통 다음이다.

- 누군가 unblock되어야 하는데 계속 blocked.
- semaphore up/down 짝이 맞지 않음.
- parent가 child load completion을 기다리는데 child가 signal하지
- file lock을 잡은 채 exit path로 가서 lock release 누락.
- interrupt가 꺼진 채 복구되지 않음.
- busy wait가 CPU를 잡아먹음.

timeout에서는 "마지막으로 살아있는 thread들이 각각 어디서 block되어 있는가"를 backtrace와 thread list가 중요하다.

11. 구현 체크리스트

Project 1 Alarm Clock 체크리스트

- struct thread에 wakeup tick 필드가 있는가?
- sleep list가 초기화되는가?
- timer_sleep(0)과 음수 tick에서 block하지 않는가?
- timer_sleep()에서 interrupt off 후 list 삽입과 thread_bloc
- thread_block() 후 interrupt level을 복구하는가?
- timer_interrupt()에서 깰 thread를 모두 unblock하는가?
- 순회 중 list_remove() 후 next pointer를 올바르게 처리하는가?
- interrupt handler에서 sleep 가능한 lock을 잡지 않는가?
- ordered sleep list라면 comparator 방향이 맞는가?

Project 1 Priority 체크리스트

- ready list가 effective priority 내림차순인가?
- thread_unblock() 후 preemption이 일어나는가?
- thread_yield()가 current를 ordered insert하는가?
- sema_up()이 highest priority waiter를 깨우는가?
- cond_signal()이 highest priority waiter를 선택하는가?
- priority donation용 list element를 기존 elem과 분리했는가?
- lock acquire에서 nested donation이 전파되는가?
- lock release에서 해당 lock 관련 donation만 제거되는가?
- base priority와 effective priority를 구분하는가?
- thread_set_priority()가 donation 중인 경우를 올바르게 처리하는가?

Project 2 체크리스트

- command line에서 executable name과 arguments를 분리하는가?
- user stack에 문자열과 argv pointer를 올바른 주소로 배치하는가?
- if_->R.rdi, if_->R.rsi를 설정하는가?
- system call number를 f->R.rax에서 읽는가?
- 4번째 syscall argument를 r10에서 읽는가?
- 모든 user pointer를 검증하는가?
- buffer가 page boundary를 넘는 경우도 처리하는가?
- invalid pointer에서 kernel panic 대신 process exit가 일어나는
- fd 0/1을 특별 처리하는가?
- file system global lock을 사용하는가?
- wait가 direct child만 허용하는가?
- child exit status가 parent wait까지 살아 있는가?
- executable file에 deny write를 적용하고 exit 때 해제하는가?
- fork에서 parent가 child 복제 성공/실패를 기다리는가?

Project 3 체크리스트

- struct supplemental_page_table이 초기화되는가?
- SPT key가 page-aligned virtual address인가?
- spt_find_page()가 unaligned address도 올바르게 찾는가?

- vm_alloc_page_with_initializer()가 type별 initializer를
- lazy load aux lifetime이 fault 시점까지 유효한가?
- vm_claim_page()가 page table mapping을 만든 뒤 swap in하는가
- frame table이 모든 allocated frame을 추적하는가?
- eviction victim 선택 중 lock이 안전한가?
- dirty/accessed bit를 올바르게 확인/갱신하는가?
- anonymous page swap slot을 bitmap으로 관리하는가?
- file-backed page가 dirty일 때만 write back하는가?
- stack growth heuristic이 너무 넓거나 좁지 않은가?
- mmap range overlap을 거부하는가?
- process exit에서 SPT의 모든 page를 destroy하는가?

Project 4 체크리스트

- inode가 direct/indirect/double indirect block을 계산하는가?
- file extension 중 실패 rollback을 고려했는가?
- hole read가 zero를 반환하는가?
- directory .와 ..를 올바르게 만드는가?
- absolute/relative path parsing이 분리되어 있는가?
- cwd reference count 또는 reopen 처리가 안전한가?
- symlink loop depth limit이 있는가?
- buffer cache hit/miss path가 dirty bit를 보존하는가?
- dirty cache entry가 eviction 또는 shutdown 때 flush되는가?
- cache entry별 lock으로 독립 sector 동시성이 가능한가?
- file extension은 atomic하게 보호되는가?

12. 미니 실습 문제

실습 1: list_entry 손으로 계산하기

struct thread 안에 struct list_elem elem이 있고, ready li
꺼냈다고 하자. 다음 질문에 답해보자.

- e는 thread의 시작 주소인가, elem 멤버의 주소인가?
- list_entry(e, struct thread, elem)는 어떤 주소를 반환하는가?
- elem 하나를 ready list와 donation list에 동시에 넣으면 왜 위험한가?

답:

- e는 elem 멤버의 주소다.
- list_entry는 elem 주소에서 offsetof(struct thread, elem)만 얻는다.
- list link의 prev, next가 하나뿐이라 두 list가 서로의 연결을 덮어쓴다.

실습 2: timer_sleep 실패 원인 찾기

증상: thread_block() assertion이 실패한다.

확인할 것:

- timer_sleep()에서 intr_disable()을 호출했는가?
- thread_block() 직전 interrupt level이 INTR_OFF인가?
- interrupt context에서 timer_sleep()을 호출하고 있지는 않은가?

증상: alarm-multiple timeout.

확인할 것:

- timer_interrupt()가 sleep list를 순회하는가?
- wakeup tick 비교가 <= ticks인가?
- list_remove() 후 다음 element 이동이 올바른가?
- thread_unblock()이 호출되었는데 ready list priority orderin

실습 3: syscall register 매핑

write(1, buffer, size)가 호출되었다. syscall handler에서 확인할

- syscall number: f->R.rax
- fd: f->R.rdi
- buffer: f->R.rsi
- size: f->R.rdx
- return value: f->R.rax

4번째 인자가 있는 syscall이면 f->R.r10을 사용한다.

실습 4: page fault 분류

다음 page fault가 처리 가능한지 생각해보자.

- lazy loaded executable page에 첫 접근: 가능. SPT에서 page를 찾

- NULL pointer read: 불가능. process kill.
- stack pointer 8 bytes 아래 write fault: stack growth 후
- kernel address를 user process가 접근: 불가능.
- read-only code page에 write: COW가 아니라면 불가능.
- swapped out anon page 접근: 가능. swap in.

13. 용어집

- alarm clock: 일정 tick 동안 thread를 sleep시키는 기능.
- busy waiting: 실제로 할 일이 없는데 반복문으로 조건을 계속 확인하며 CPU를 쓰는
- condition variable: lock과 함께 조건이 변하기를 기다리는 동기화 primi
- donation: 높은 priority thread가 lock holder에게 priority
- effective priority: donation이 반영된 실제 scheduling prio
- frame: physical memory의 page-sized block.
- intrusive list: element link가 데이터 구조체 안에 들어가는 list 방
- kernel thread: kernel address space에서 실행되는 thread.
- lazy loading: 실제 접근이 발생할 때까지 page 내용을 로드하지 않는 방식.
- lock: owner가 있는 mutual exclusion primitive.
- mmap: file 내용을 process address space에 mapping하는 기능.
- page: virtual memory의 page-sized 영역.
- page fault: page table translation 또는 permission 검사
- page table: virtual address를 physical address로 변환하는
- PML4: x86-64 4-level page table의 최상위 table.
- ready list: 실행 가능한 thread를 담는 scheduler queue.
- semaphore: counter 기반 synchronization primitive.
- SPT: supplemental page table. page fault 처리를 위한 kern
- swap: physical memory에서 밀려난 anonymous page 내용을 disk에
- system call: user program이 kernel service를 요청하는 경계.
- user pointer: user address space 안의 pointer. kernel이

14. 참고 자료

공식 자료:

- KAIST Pintos Manual Introduction: https://casys-kais
- Getting Started: https://casys-kaist.github.io/pintos-kaist/introduct
- Project 1 Alarm Clock: https://casys-kaist.github.io/pintos-kaist/project1/
- Project 1 Priority Scheduling: https://casys-kaist.github.io/pintos-kaist/project1/
- Project 2 Argument Passing: https://casys-kaist.github.io/pintos-kaist/project2/
- Project 2 User Memory: https://casys-kaist.github.io/pintos-kaist/project2/
- Project 2 System Calls: https://casys-kaist.github.io/pintos-kaist/project2/
- Project 3 Memory Management: https://casys-kaist.github.io/pintos-kaist/project3/
- Project 3 Stack Growth: https://casys-kaist.github.io/pintos-kaist/project3/
- Project 3 Swap In/Out: https://casys-kaist.github.io
- Project 4 Synchronization: https://casys-kaist.github.io/pintos-kaist/project4/

- Appendix Threads: https://casys-kaist.github.io/pint
- Appendix Synchronization: https://casys-kaist.github.io/pintos-kaist/appendix/
- Appendix Debugging Tools: https://casys-kaist.github.io/pintos-kaist/appendix/
- Appendix Hash Table: https://casys-kaist.github.io/p

현재 repo에서 직접 확인한 주요 파일:

- README.md
- pintos/README.md
- pintos/devices/timer.c
- pintos/include/devices/timer.h
- pintos/threads/thread.c
- pintos/include/threads/thread.h
- pintos/threads/synch.c
- pintos/include/threads/synch.h
- pintos/include/threads/interrupt.h
- pintos/include/lib/kernel/list.h
- pintos/include/lib/kernel/hash.h
- pintos/userprog/process.c
- pintos/userprog/syscall.c
- pintos/include/lib/syscall-nr.h
- pintos/include/vm/vm.h
- pintos/vm/vm.c
- pintos/include/vm/uninit.h
- pintos/include/vm/anon.h
- pintos/include/vm/file.h

15. 마지막 조언

Pintos에서 가장 위험한 습관은 "일단 돌아가게만"이다. 커널은 작은 우연으로도 며칠 뒤 테스트에서 터진다. 대신 매번 다음 네 문장을 확인하자.

- 이 자료구조의 owner는 누구인가?
- 이 포인터나 object의 lifetime은 어디서 끝나는가?
- 이 코드는 interrupt context에서 실행될 수 있는가?
- 이 함수가 끝났을 때 반드시 유지되어야 하는 불변식은 무엇인가?

이 네 질문에 답할 수 있으면 구현은 느려도 단단해진다. Pintos는 결국 큰 퍼즐이지만, 모양은 꽤 정직하다.